EPub-InkPlate - Installation Guide

Important Notes

Please Note: If you are updating from a version prior to V3.0.0 of the application, the SD-Card must be re-initialized. See the section on preparing the SD-Card for details.

Updating from V3.0.0-BETA

Updating from V3.0.0-BETA will require, to the minimum, doing the following on the SD-Card:

- Remove all `.locs` and `.toc` files from the `/books` folder.
- Remove the `/fonts` folder, and copy the `/fonts` folder from the `/SDCard/fonts` distribution.

Introduction

This document describes the installation procedure for the EPub-InkPlate application, which can be adapted to suit your requirements.

The installation consists of

- Preparing your computer with the proper applications needed for the installation process.
- Preparing an SD-Card with the appropriate information.
- Uploading the application to the InkPlate device.

The latest binaries for the InkPlate are available in release bundles on the application's GitHub project page: "https://github.com/turgu1/EPub-InkPlate/releases". This procedure describes installation using the *esptool* upload tool, which is the simplest approach because it does not require a full development environment.

If you want to build the application yourself, PlatformIO is no longer used. You must install Espressif's development tool suite, including ESP-IDF and the Espressif VS Code extension, and build the project from that environment. The following procedure uses the *esptool* instead of the Espressif's tool suite, sufficient for the need of installing a EPub-InkPlate distribution.

Prerequisites

esptool is a Python program used to upload applications to ESP32 (or ESP8266) devices. It requires *Python* 3.7 or newer. First, ensure that *Python* and *pip* are installed on your computer (see: "https://wiki.python.org/moin/BeginnersGuide/Download"). Then, on Windows, Linux, or macOS, install esptool by running the following command in a shell window:

```
pip install esptool
```

For more information, consult: "https://docs.espressif.com/projects/esptool/en/latest/esp32/installation.html".

The InkPlate device uses a CH340 USB-to-UART converter. This device is natively supported on MacOS and Linux. If your computer does not have a CH340 driver installed, you will need to install one. See the following page for the appropriate procedure: "https://learn.sparkfun.com/tutorials/how-to-install-ch340-drivers/all".

Next, download the release from the GitHub repository at: "https://github.com/turgu1/EPub-InkPlate/releases". The file to download is **release-X.X.X-inkplate...zip**, found in the `Assets` section below the release description. Extract its contents — you will find two folders (`bin` and `SDCard`), along with the installation document and the user's guide in PDF format.

Preparing the SD-Card

The SD-Card must be formatted with a FAT32 (MS-DOS/VFAT) partition, which is typically the case for new cards. Do not use ExFAT, as it is not supported by the application. The release's `SDCard` folder contains everything needed to initialize the card. Simply copy the folder's contents (including all sub-folders) to the card.

The config.txt file The `config.txt` file in the card's root folder can be edited to set your Wi-Fi parameters (`wifi_ssid`, `wifi_pwd`, `http_port`), timezone (`tz`), and Internet time server address (`ntp_server`). Because these parameters contain free-form text or numeric values, they cannot be edited through the EPub-InkPlate application's menus — they must be set manually in this file. The file is loaded at startup.

The Wi-Fi parameters allow the application to access your local network, enabling you to manage the book collection on the card from a web browser. This is optional — you can always update the SD-Card contents by inserting it directly into your computer.

Wi-Fi is also used to synchronize the device clock with an Internet time server. A default NTP server address is pre-configured in the `ntp_server` parameter and should be sufficient for most users, but you may substitute your own. Note that clock synchronization must be started manually from within the application. See the User's Guide for details.

The `tz` parameter is a specially formatted string (called the *Posix Time Format*)that tells the application how to convert the internal UTC clock to local time. A file named `timezones.csv` is included with the EPub-InkPlate distribution and contains timezone values for 461 cities worldwide. For example, the timezone value for Zagreb is `CET-1CEST,M3.5.0,M10.5.0/3`. The full format specification is documented at: "https://sourceware.org/glibc/manual/latest/html_node/TZ-Variable.html". This table has been built from the current Ubuntu timezone database dated 2026.05.25 and may change as countries modify their way of dealing with daylight saving time. Every time a new distribution of EPub-InkPlate is generated, this file is also re-generated.

`config.txt` also contains parameters that are managed through the application's menus. Do not modify these by hand. If they are deleted from the file, the application will restore them to their default values.

Your books As mentioned in the previous section, you can add books to the device through the built-in web server using a browser. See the User's Guide for instructions on starting the server.

Alternatively, you can add books manually. All books must be in EPub format with a lowercase `.epub` file extension, placed in the `books` folder on the SD-Card. Insert the SD-Card into your computer to do this directly.

If the SD-Card has been used by the application before, you may find additional files in the `books` folder. These are used to track your reading progress and interaction with each book. If deleted, the application will recreate them as needed.

Once finished, insert the card into the device.

Uploading the application program

The release's `bin` folder contains the application, bootloader, and partition binaries. To upload them, connect the device to a USB port, turn it on, change your working directory to the `bin` folder, and run the appropriate command:

On Linux or MacOs (in a shell window):

```
$ sh upload.sh
```

On MS Windows:

```
.\upload.bat
```

Here is an example output of the execution:

```
$ sh upload.sh
esptool.py v3.0
Serial port /dev/ttyUSB0
Connecting......
Chip is ESP32-D0WDQ6 (revision 1)
Features: WiFi, BT, Dual Core, 240MHz, VRef calibration in efuse,
  > Coding Scheme None
Crystal is 40MHz
MAC: fc:f5:c4:1b:4e:cc
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 230400
Changed.
Configuring flash size...
Auto-detected Flash size: 4MB
Compressed 25136 bytes to 15148...
Wrote 25136 bytes (15148 compressed) at 0x00001000 in 0.7 seconds
  > (effective 297.9 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 143...
Wrote 3072 bytes (143 compressed) at 0x00008000 in 0.0 seconds
  > (effective 2244.8 kbit/s)...
Hash of data verified.
Compressed 1086128 bytes to 554716...
Wrote 1086128 bytes (554716 compressed) at 0x00010000 in 24.9 seconds
  > (effective 348.5 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

```
$
```

Once the upload is complete, the device will automatically reboot. Refer to the User's Guide for information on how to use the application.

Depending on your computer, you may need to adjust some options in the upload script:

- The USB device is expected to appear as `/dev/ttyUSB0` on Linux (e.g., Linux Mint, Ubuntu) and macOS, or as `COM3` on Windows. If your device uses a different name, locate it and update `upload.sh` (Linux/macOS) or `upload.bat` (Windows) accordingly.
- If the upload fails due to a speed issue, reduce the baud rate in `upload.sh` (or `upload.bat`). The default is **230400**; you can lower it to **115200** or less.